

WebTrail 项目答辩 Q&A

Q1：为什么选择纯 Python 标准库，不用任何第三方依赖？

取证工具的核心要求是**可审计、可移植、零部署成本**。

第三方库意味着需要 `pip install`，在某些隔离环境（如涉密内网）根本无法联网。标准库的代码任何人都能审查，不会有供应链攻击风险。而且 Python 标准库已经提供了我们需要的全部能力——`sqlite3` 读数据库、`tkinter` 做 GUI、`json` / `csv` 做输出、`hashlib` 做哈希、`threading` 做异步。

Q2：你的证据哈希链是怎么设计的？如果有人说"你这哈希是提取之后才算的"你怎么反驳？

哈希计算发生在**提取之前**，代码在 `pipeline.py` 的 `collect_evidence_hashes` 中。

流程是：`collect_evidence_hashes` → `run_extraction` → `run_profiling`，哈希是第一阶段的独立步骤。哈希结果写入报告的 `metadata.file_hashes` 字段，报告本身又生成 `report_checksums.json` 自校验。

如果你质疑哈希的时机，可以看报告中的 `extraction_time` 时间戳和文件系统的 `mtime` 交叉验证——哈希时刻一定早于提取时刻。

Q3：Edge 为什么要继承 Chrome，而不是独立实现？这不违反"组合优于继承"吗？

Edge 使用 Chromium 内核，其浏览器数据的**物理格式**和 Chrome 100% 一样——SQLite 表结构、JSON 结构、时间戳编码完全一致。

「组合优于继承」针对的是行为复用，但这里复用的是**完全相同的实现逻辑**。如果用组合，需要把 Chrome 的 212 行代码包装成一个可注入的策略对象，Edge 再组合它——多一层间接性，却没有增加任何灵活性。

GoF 设计模式里有一个明确的原则：**当两个类有 "is-a" 关系且行为完全一致时，继承是正确的选择**。这里 Edge 就是一个 Chromium 浏览器，它和 Chrome 的区别仅仅是数据目录不同和浏览器名称不同。

Q4：如果浏览器正在运行，SQLite 被锁定怎么办？

我们设计了双层回退机制，在 `sqlite_utils.py` 的 `safe_connect_with_fallback` 中：

1. 先尝试 `mode=ro` 只读连接，大部分情况下只读不触发锁冲突
2. 如果失败，调用 `copy_to_temp` 将数据库文件**复制**到临时目录
3. 在临时文件上建立普通连接读取
4. 关闭连接时，通过猴子补丁的 `conn.close` 自动 `unlink` 删除临时文件

关键点：复制而非移动，不修改原始证据文件；临时文件自动清理，不残留。

Q5：你的时间戳处理有什么坑？怎么解决的？

三个坑。

坑1：Chrome 用 1601-01-01 纪元微秒（Windows FILETIME），Firefox 用 1970-01-01 纪元微秒（PRTime），两者差 369 年。必须各写专用转换函数。

坑2：所有浏览器存的都是 UTC 时间，但活跃时段分析需要本地时间。最初我用字符串切片 `ts[11:13]` 直接取 UTC 小时，导致中国用户的高峰时段差了 8 个小时。修复方式是 `datetime.fromisoformat` 解析 ISO 字符串 → `astimezone()` 转本地时区 → 取小时。

坑3：`datetime.fromisoformat` 是 Python 3.7 引入的，3.11 之前不支持 `Z` 后缀。我们的时间戳统一用 `+00:00` 格式输出，保证全版本兼容。

Q6：你的程序怎么保证不污染原始证据？

四个层面。

连接层：所有 SQLite 连接带 `mode=ro` 和 `PRAGMA query_only=ON`，数据库层面禁止写入。

文件层：锁定回退时用 `shutil.copy2` 复制到临时文件，不移动、不修改原始文件。

编码层：JSON 文件（如书签、Firefox logins.json）只读取，不写回。

流程层：哈希在提取前完成，提取过程中任何步骤失败都有 `errors` 列表记录，不会静默篡改。

Q7：为什么要同时输出 JSON 和 CSV 两种格式？

JSON 给程序读，CSV 给人读。

JSON 包含嵌套的完整画像结构、元数据和哈希链，适合后续用脚本二次分析或导入数据库。CSV 是扁平化表格，UTF-8 BOM 编码确保 Excel 双击不乱码，适合调查员直接浏览、筛选、排序。

各司其职，互不替代。

Q8：你的错误隔离是怎么做的？举个例子。

每个浏览器适配器的 `run()` 方法内部用 try/except 包裹每个 `extract_*` 调用。

```
for extract in [self.extract_history, self.extract_cookies, ...]:
    try:
        records += extract(profile_path)
    except Exception as e:
        errors.append(str(e))
```

比如 Firefox 的 `logins.json` 损坏导致 `extract_logins` 抛异常，Firefox 的历史和 Cookie 提取不受影响，Chrome 和 Edge 的提取更不受影响。最终 `ExtractionResult.errors` 列出所有异常，调查者可以看到哪些模块失败了，但不会丢失其他数据。

Q9：风险检测的"隐私模式推断"是怎么做的？有什么局限？

逻辑很简单——如果全量提取记录少于 10 条，标记 `private_mode_hint = True`。

前提是三个浏览器加起来也凑不够 10 条，说明用户长期使用无痕模式。无痕模式下 Chrome 不写 History、Firefox 不写 places.sqlite。

局限：刚装系统、新建用户、或者浏览器第一次使用也会有同样现象。所以这个指标叫 `hint`（暗示）而非 `detected`（检测到），需要结合其他指标交叉验证。

Q10：如果让你扩展这个项目，加一个功能，你会加什么？怎么实现？

我会加**时间线可视化**——用浏览器打开一个独立的 HTML 页面，展示交互式时间线。

实现方案：在 `output/writer.py` 中新增一个方法，用 `plotly.js` 或纯 Canvas 渲染一个自包含的 HTML 文件（不依赖外部 CDN）。X 轴是时间，每个点是一个浏览事件，颜色区分类型，可以缩放、悬停看详情。

因为 HTML 是纯文本，不影响现有的 JSON/CSV 输出链，也不引入新依赖。

Q11：你的代码里有没有使用多线程？为什么？

有，但只在 GUI 里用。

```
thread = threading.Thread(target=self._run_analysis, daemon=True)
thread.start()
```

命令行模式不需要多线程，因为用户就是在等结果。但 GUI 模式下，分析任务如果在主线程跑，tkinter 的事件循环会被阻塞，窗口"假死"。

单独的线程跑分析，通过 `root.after()` 把 UI 更新扔回主线程，这是 Python GUI 编程的标准做法。

`daemon=True` 确保关闭窗口时线程自动退出。

Q12：你对这个项目最满意的一个设计决策是什么？

三阶段管道设计。 `collect_evidence_hashes` → `run_extraction` → `run_profiling` 这个顺序是固定的、不可跳过的，由 `pipeline.py` 强制保证。

很多工具把"提取"和"分析"混在一起，哈希更是可有可无。我们把三个关注点彻底分离，每个阶段的输出是下一阶段的输入，CLI 和 GUI 共享完全相同的管道代码——`gui/app.py` 和 `main.py` 都调 `pipeline` 里的函数，只是呈现方式不同。这就是"关注点分离"在架构层面的体现。

Q13：Cookie 里有 `.google.com` 这种前导点，你怎么处理的？

Cookie 的 `host_key` 字段按 RFC 规范带前导点表示域级 Cookie，如 `.google.com`。

在 `url_utils.py` 的 `extract_domain` 中加了一行 `.rstrip(".")`，去掉前导点再提取域名，这样 `.google.com` 和 `google.com` 就会统一归并，不会在域名排名里出现两条。

这个 bug 是实际跑数据时发现的——TOP 域名里同时出现了 `.youtube.com` 和 `youtube.com`，修起来就一行代码。